

CSE 314 — IPC Assignment 3 (Peaky Blinders) :

Explanation ও Viva Prep

Student ID: 2205047 | Sir-কে বোঝানোর জন্য প্রস্তুত করা নোট (code বাদে)

১. High-Level Explanation — পুরো Assignment-টা কী চাচ্ছে

Story/Scenario (শুধু context)

Peaky Blinders থিমে র‍্যাপ করা একটা classic OS synchronization problem — আসল যা বোঝা দরকার:

N জন **operative** আছে, তাদেরকে **M** জনের **group**-এ ভাগ করা হয়েছে (মোট N/M টা group)। প্রতিটা group-এর সর্বোচ্চ **ID**-ওয়ালা **member**-ই সেই **group**-এর **"leader"**। পুরো কাজ দুইটা ধাপে (Phase) হয়:

Phase 1 — Document Recreation (Task 1: Typewriting Station synchronization)

- প্রতিটা operative এলোমেলো সময়ে (random delay দিয়ে) এসে ৪টা **typewriting station**-এর একটাতে (TS1–TS4) কাজ করতে যায় — কোন station, সেটা $(ID \bmod 4) + 1$ সূত্র দিয়ে ঠিক হয়।
- একটা station-এ একবারে একজনই কাজ করতে পারবে — তাই station busy থাকলে বাকিদের wait করতে হবে (কিন্তু busy-waiting/polling করে না, blocked হয়ে থাকে)।
- কাজ শেষ হলে সেই operative group leader-কে জানায় (physically কিছু "পাঠানো" হয় না, শুধু একটা shared counter বাড়ানো হয়)।
- যখন group-এর সবাই (M জনই) Phase 1 শেষ করে ফেলে, তখন leader তার নিজের Phase 2 শুরু করতে পারবে।

Phase 2 — Logbook Entry (Task 2: Reader-Writer Problem)

- Leader central logbook-এ এন্ট্রি লিখতে (write) যায়।
- সাথে সাথে দুইজন Intelligence Staff periodically logbook পড়তে (read) আসে (progress দেখার জন্য)।
- Rule: একসাথে একজনই লিখতে পারবে (writer), কিন্তু write না হলে একসাথে একাধিক জন পড়তে পারবে (readers) — এটাই classical **Readers-Writers Problem**, আর এখানে **readers**-কে **priority** দেওয়া হয়েছে (marking scheme-এও এটা explicitly চাওয়া হয়েছে)।

দুইটা মূল Implementation শর্ত

1. **No busy waiting** — কোনো thread কখনো loop-এ বসে বারবার condition check করে CPU নষ্ট করবে না; সবকিছু semaphore/mutex দিয়ে block হয়ে থাকবে।

2. সবাইকে একসাথে তৈরি করা, কিন্তু এলোমেলো সময়ে শুরু করানো — সব `pthread_create` একসাথে করা হয়, কিন্তু প্রতিটা thread নিজে থেকে একটা random (Poisson distribution থেকে নেওয়া) delay নিয়ে তারপর আসল কাজ শুরু করে।

Input/Output

Input file-এ দুই লাইন: `N M` আর `x y` (x = document recreation-এর সময়, y = logbook entry-র সময়)। Output একটা ফাইলে লেখা হয় (`std::cout` -কে redirect করে)।

২. Code-এর প্রতিটা অংশ কী কাজ করছে (block-by-block, high level)

Global variables ও Data Structure

- `station_sem[NUM_STATIONS]` — ৪টা typewriting station-এর জন্য ৪টা binary semaphore (init value = 1), প্রতিটা mutex-এর মতো কাজ করছে।
- `struct Group` — প্রতিটা group-এর জন্য `count` (কয়জন Phase-1 শেষ করেছে), `count_mtx` (সেটা protect করার মিউটেক্স), আর `leader_sem` (leader এখানে wait করে)।
- `logbook_sem` ও `read_count_mtx` ও `read_count` — Task 2-এর classic reader-writer state।
- `print_mtx` — আউটপুট লেখার সময় দুই thread-এর টেক্সট মিশে না যায়, তার জন্য mutex।
- `simulation_done` — সব operative শেষ হয়ে গেলে true হয়, staff thread-দের loop বন্ধ করার সংকেত।

Helper Functions

- `get_time_units()` — simulation শুরু হওয়ার পর থেকে কত "time unit" পার হয়েছে হিসাব করে।
- `sleep_units(units)` — নির্দিষ্ট সংখ্যক time-unit বাস্তব সময়ে ঘুমিয়ে থাকে (`usleep` দিয়ে)।
- `log_print(s)` — thread-safe প্রিন্ট: `print_mtx` লক করে তারপর `cout` -এ লেখে।
- `poisson_delay(lambda)` — Poisson distribution থেকে random delay জেনারেট করে (Randomized Arrival শর্ত পূরণ করতে)।

`init_all()`

সব semaphore/mutex initialize করে, group-এর সংখ্যা (`N/M`) হিসাব করে, আর simulation-এর "t=0" মুহূর্ত রেকর্ড করে রাখে।

`write_logbook()`

Group leader এটা কল করে — `logbook_sem` নিয়ে (writer lock), `Y` সময় ধরে "লেখার" কাজ simulate করে, `completed_operations` বাড়ায়, প্রিন্ট করে, লক ছেড়ে দেয়।

`operative_thread()` — Task 1-এর মূল লজিক

1. **Random arrival delay:** সব thread একসাথে তৈরি হয়, কিন্তু প্রতিটা নিজে থেকে একটা Poisson delay নিয়ে ঘুমিয়ে থাকে — বাস্তব জগতের মতো আলাদা আলাদা সময়ে "পৌঁছানো" simulate হয়।
2. **Station নির্ধারণ:** `id % NUM_STATIONS` দিয়ে কোন station তার জন্য নির্ধারিত সেটা বের করে।
3. **Station দখল:** `sem_wait` কল করে — খালি থাকলে সাথে সাথেই ঢুকে যায়, ব্যস্ত থাকলে সত্যিকারের block হয়ে থাকে (কোনো polling loop ছাড়াই)।
4. **কাজ:** `X` সময় ধরে ঘুমিয়ে "document recreation" simulate করে।
5. **Station ছেড়ে দেওয়া:** `sem_post` করে — পরে যে wait করছিল সে জেগে ওঠে।
6. **Group counter** বাড়ানো: mutex দিয়ে protected `count++` — যেই increment-টা count-কে ঠিক M-এ নিয়ে যায়, শুধু সেই operative-ই "everyone_done" দেখে, আর leader-কে জাগানোর জন্য `sem_post(leader_sem)` করে (এটা নিশ্চিতভাবে ঠিক একবারই ঘটে)।
7. **Leader check:** যদি নিজের ID-ই সেই group-এর leader-এর ID হয়, তাহলে `sem_wait(leader_sem)` করে অপেক্ষা করে (যদি সে নিজেই শেষ জন হয়ে থাকে, ইতিমধ্যে posted থাকায় সাথে সাথেই পাস করে যায়) — তারপর `write_logbook()` কল করে Phase 2 শুরু করে।

`staff_thread()` — Task 2-এর মূল লজিক (Reader-Writer)

- প্রতিটা staff-এর নিজস্ব Poisson mean (`lambda`) আছে — "different random intervals" শর্ত পূরণের জন্য।
- **Reader Entry:** `read_count_mtx` লক করে `read_count++` করে; যদি এই মুহূর্তে সে-ই প্রথম reader হয় (`read_count` হয় ১), তাহলে সে `logbook_sem` (writer lock) নিয়ে নেয় — যাতে কোনো writer ভেতরে ঢুকতে না পারে।
- এরপর নিরাপদে পড়ে এবং প্রিন্ট করে।
- **Reader Exit:** `read_count--` করে; যদি সে-ই শেষ reader হয় (`read_count` হয় ০), তাহলে `logbook_sem` ছেড়ে দেয় — writer আবার ঢুকতে পারবে।
- `while(!simulation_done)` লুপ — কিন্তু প্রতিবার ভেতরে ঘুমিয়ে থাকে (busy-wait না), `main()` থেকে flag true হলে ঘুম থেকে উঠে খামে।

`main()`

- Input file থেকে `N M X Y` পড়ে।
- `cout.rdbuf()` ব্যবহার করে `cout` -কে output file-এ redirect করে দেয় — এরপর যেখানেই `cout <<` লেখা আছে, সেটা console-এ না গিয়ে ফাইলে যায়।

- সব N-টা operative thread একটানা লুপে তৈরি করে (একসাথে "জন্ম" নেয়), তারপর ২টা staff thread তৈরি করে।
- সব operative thread শেষ হওয়ার জন্য অপেক্ষা করে (`pthread_join`), তারপর `simulation_done = true` সেট করে staff thread-দের খামার সংকেত দেয়, তাদেরও join করে।
- শেষে `cout` -কে আবার console-এ ফিরিয়ে দেয়।

⚠ একটা ছোট **formatting bug** (demo-র আগে জেনে রাখা ভালো)

Output-এ দেখা যায়:

Operative 4 has arrived at typewriting station TS1at time 2 — "TS1" আর "at time"-এর মাঝে space নেই।

কারণ: `"... station TS" + to_string(station_number + 1) + "at time "`
`+ ...` — আগের অংশের শেষে space নেই। এটা logic bug না, শুধু presentation —
`"at time "` -এর আগে একটা space (`" at time "`) যোগ করলেই ঠিক হয়ে যাবে।

৩. Sir তোমাকে যা যা জিজ্ঞেস করতে পারেন — এবং উত্তর

Task 1 (Typewriting Station) নিয়ে

Q: `station_sem` আসলে কী, আর কেন এটাকে **semaphore** বলা হচ্ছে যদি এটা **mutex**-এর মতো কাজ করে?

A: এটা একটা **binary counting semaphore** (init value = 1)। value শুধু 0/1-এর মাঝে থাকায় **mutex**-এর মতো আচরণ করছে — একে "binary semaphore" বলা হয়।

Q: একজন **operative station busy** পেলে কী করে? **Busy-waiting** এড়ানো হলো কীভাবে?

A: `sem_wait()` কল করে — value 0 হলে thread সত্যিকারের block হয়ে যায়, OS ঘুম পাড়িয়ে রাখে, কোনো loop করে check করে না। `sem_post()` হলে OS নিজেই একজন waiting thread-কে জাগায়।

Q: `station_number = id % NUM_STATIONS` — এটা কীভাবে **"(ID mod 4) + 1"** সূত্রের সাথে মিলছে?

A: `station_number` 0-indexed (0–3) রাখা হয়েছে, প্রিন্ট করার সময় `+1` করে 1-indexed (TS1–TS4) দেখানো হচ্ছে। উদাহরণ: operative 4 → $4\%4=0$ → TS1 (spec-এর উদাহরণের সাথে মিলছে)।

Q: একটা group-এর সবাই কবে Phase 1 শেষ করেছে সেটা কীভাবে জানা যায়? Race condition হবে না তো?

A: প্রতিটা group-এর নিজস্ব `count`, mutex দিয়ে protected। শুধু যেই increment count-কে ঠিক M-এ নেয়, সেই operative-ই "everyone_done" দেখে — leader-কে জাগানোর post ঠিক একবারই হয়।

Q: Leader নিজেই যদি group-এর শেষ সদস্য হয়, deadlock হবে না তো?

A: না। সে নিজেই সেই মুহূর্তে `sem_post(leader_sem)` করে ফেলে। পরে নিজের `sem_wait(leader_sem)` -এ পৌঁছালে semaphore ইতিমধ্যে posted থাকায় সাথে সাথেই পাস করে যায়।

Task 2 (Reader-Writer / Logbook) নিয়ে

Q: কোন ধরনের reader-writer solution implement করেছে — readers-priority নাকি fair?

A: **Readers-priority (first-reader/last-reader)** solution। প্রথম reader লেখার lock নেয়, বাকিরা সরাসরি চুকে যায়। শেষ reader lock ছাড়ে। Writer অনেকক্ষণ starve করতে পারে — এটাই readers-priority-র বৈশিষ্ট্য, marking scheme-এও এটাই চাওয়া হয়েছে।

Q: read_count প্রোটেক্ট করতে আলাদা read_count_mtx কেন লাগলো?

A: `logbook_sem` শুধু writer-এর exclusive access নিয়ন্ত্রণ করে। কিন্তু `read_count` প্রতিটা reader increment/decrement করে — এটা আলাদা protect না করলে দুইজন reader একসাথে আপডেট করলে race condition হবে।

Q: দুইজন staff-এর জন্য আলাদা lambda (3.0 আর 5.0) কেন?

A: Assignment-এ explicitly বলা আছে "different random intervals for the two intelligence staff members" — দুইজন independent, ভিন্ন গতিতে চেক করছে সেটা দেখানোর জন্য।

Q: completed_operations কি thread-safe ভাবে read/write হচ্ছে?

A: এটা শুধু `write_logbook()`-এর ভেতরেই increment হয় (`logbook_sem` দিয়ে protected)। Staff থ্রেডরা শুধু পড়ে, লেখে না — readers-writers lock-এর কারণে reader আর writer কখনো একসাথে থাকে না, তাই এই read/write নিরাপদ।

General / Design নিয়ে

Q: Poisson distribution কেন ব্যবহার করেছে?

A: বাস্তব জগতের "random arrival events" মডেল করার জন্য standard পরিসংখ্যানিক পদ্ধতি — assignment নিজেই এটা ব্যবহার করতে বলেছে।

Q: UNIT_MS = 100 মানে কী, কেন বেছে নিলে?

A: "1 time unit" = 100 millisecond বাস্তব সময়। যথেষ্ট ছোট রাখা হয়েছে যাতে demo দ্রুত শেষ হয়, আবার যথেষ্ট বড় যাতে event-এর ordering স্পষ্ট দেখা যায়।

Q: simulation_done কে volatile করা হয়েছে কেন? এটা কি thread-safe?

A: volatile compiler-কে বলে বারবার memory থেকে re-read করতে (cache না করে)। কিন্তু C++ standard অনুযায়ী এটা strictly thread-safety/memory-ordering গ্যারান্টি দেয় না — টেকনিক্যালি সঠিক উপায় `std::atomic<bool>`। এটা x86-এ practically কাজ করে, কিন্তু "best practice" না — উন্নতির একটা জায়গা হিসেবে বলা যায়।

Q: Staff threads কীভাবে শেষ হয়? এটা কি busy-waiting?

A: না। প্রতিটা loop iteration-এ থ্রেড একটা random সময় ধরে সত্যিই ঘুমিয়ে থাকে (spin করে না), তারপর একবার flag check করে। সব operative শেষ হলে main() simulation_done = true করে, staff থ্রেড ঘুম থেকে জেগে সেটা দেখে বেরিয়ে আসে।

Q: cout.rdbuf() দিয়ে কী করা হচ্ছে?

A: cout-এর "গন্তব্য" বদলে ফাইলে ঘুরিয়ে দেওয়া হচ্ছে — এরপর কোডের যেকোনো cout << ... আসলে output file-এই লেখা হয়। শেষে আবার console-এ ফিরিয়ে দেওয়া হয়।

Q: Deadlock হওয়ার কোনো সম্ভাবনা আছে কি?

A: না। প্রতিটা lock/semaphore-এর জন্য নিশ্চিত release path আছে, কোনো nested/circular locking নেই — তাই circular-wait condition তৈরি হয় না।

Q: মোট কয়টা thread তৈরি হয়?

A: N-টা operative thread + ২টা staff thread = N + 2 টা।